

Portion fabricant vs quantite consommee

Force Tracker — trace du 10/09/2026, sur la version en ligne **ft-v1189**. Reponse au cahier « BUG A ISOLER ». **Rien n'est corrige** : la consigne etait « ne corrige rien avant mesure ».

EN UNE PHRASE

Le bug est confirme, et il est plus large que les trois cas signales. La portion du fabricant n'est pas seulement affichee : elle est **reellement ecrite** dans le journal. Et le probleme ne touche pas « les produits ayant une portion fabricant » — il touche **tous les produits scannes**, y compris ceux qui n'en declarent aucune. **La cause tient en une ligne** : la condition qui ecrit la quantite ne demande jamais si un geste a eu lieu.

CE QUI EST MESURE, ET CE QUI NE L'EST PAS

Chaque chiffre vient d'un **vrai navigateur** (Chromium + Playwright) qui appelle `_lookupBarcode` avec une fiche produit servie par un faux reseau, puis relit ce que l'application **ecrit** dans `S.foodLog`. Le geste joue est exactement celui du rapport : **on scanne, on ne touche a aucune quantite, on enregistre**. **Limite** : pas de WebKit ici, et aucun acces au reseau Open Food Facts — les fiches sont donc **fabriquees**, en ne faisant varier que le champ etudie.

1. Le champ source

C'est `serving_quantity`, publie par la fiche Open Food Facts. Rien d'autre : faire varier ce seul champ suffit a faire varier le resultat, toutes choses egales par ailleurs.

produit	serving_quantity envoye	champ affiche	q ENREGISTRE
Raynal	205	205	q:205 · u:'g'
Thon	140	140	q:140 · u:'g'
Cassegrain	187.5	187,5	q:187.5 · u:'g'

LE 187,5 EST LA PREUVE QUE LA VALEUR N'EST PAS CALCULEE

Aucun calcul de l'application ne produit une decimale. Ce n'est pas « la boite de 410 g divisee par 2 » : le poids du paquet est lu separement par `_offPoidsPaquet`, qui rend bien **410** et l'affiche dans sa propre pastille. *Que 205 soit la moitie de 410 est une coincidence de ce produit-la.*

2. La fonction qui l'injecte

`_offRemplirFormulaire`, et le repli a 100 est dans la meme expression :

```
const serv=parseFloat(p.serving_quantity)||0;
const g=serv>0?serv:100;
const gramsEl=document.getElementById('af-bc-grams');if(gramsEl)gramsEl.value=g;
```

Ce chemin est **commun a plusieurs portes** : le scan, la recherche par nom, les produits de marque et CIQUAL passent tous par cette fonction. Elle est appelee avec `serving_quantity:0` pour CIQUAL et l'etiquette recopiee, qui retombent donc sur le 100.

3. Visuel, ou reellement ecrit ?

REELLEMENT ECRIT

Ce n'est **pas** un simple pre-remplissage d'ecran. Apres le geste « scanner, ne rien toucher, enregistrer », la ligne du journal porte `q:140, u:'g'`. La valeur descend jusqu'a la donnee, est persistee, part au cloud, et sert de reference a toutes les reprises ulterieures de cet aliment.

4. Sur quels produits ? — plus large que le rapport

La question posée était « est-ce que cela arrive sur tous les produits ayant une portion fabricant ». **La réponse mesurée est : sur tous les produits scannes, portion ou pas.**

cas	champ affiche	q ENREGISTRE	d'ou vient le nombre
portion declaree (205 / 140 / 187,5 / 30)	la portion	ecrit	serving_quantity de la fiche
aucune portion declaree	100	q:100 · u:'g'	value="100" écrit en dur dans le HTML
portion declaree = 0	100	q:100 · u:'g'	idem

LE CAS SANS PORTION EST LE PLUS DIFFICILE A DEFENDRE

Une portion fabricant est au moins **une information publiée par quelqu'un** — discutable, mais réelle. Le **100**, lui, ne vient de personne : c'est une valeur par défaut de mise en page. **La règle que tu énonces le couvre exactement** : une valeur de référence, une suggestion ou une portion fabricant ne doit jamais devenir silencieusement un choix utilisateur. Le 100 n'est même pas une suggestion.

5. Pourquoi l'application considère cette valeur comme un choix

Parce qu'elle ne pose jamais la question. La condition qui écrit la quantité, dans `_provFood` :

```
const row=document.getElementById('af-bc-row');
const g=numFR((document.getElementById('af-bc-grams')||{}).value)||0;
if(row&&row.style.display!='none'&&_bcNutr&&g>0){ p.q=g; p.u='g'; }
```

Elle vérifie **trois choses** : le bloc est-il visible, une fiche produit est-elle chargée, le champ contient-il un nombre supérieur à zéro. **Aucune des trois ne dit quoi que ce soit sur un geste de la personne**. Un champ pré-rempli par la fiche et un champ tape à la main sont, à cet endroit, **strictement indiscernables**.

ET CE N'EST PAS UNE IDÉE QUI MANQUE : C'EST UN DRAPEAU QUE L'APPLICATION POSSEDE DÉJÀ DEUX FOIS

Deux autres blocs de ce même écran résolvent exactement ce problème, et ils le résolvent de la même façon : un drapeau qui n'est posé que par un **clic ou une frappe**.

bloc de l'écran d'ajout	drapeau de geste	mesure
poids declare a la main	<code>_afPoidsPose</code>	EXISTE
boutons de portion (1/2 · 1 · 2 · 3)	<code>_afPortionPose</code>	EXISTE
bloc du SCAN (af-bc-grams)	—	AUCUN

LE BLOC DU SCAN EST LE SEUL DES TROIS A NE PAS SAVOIR

Les deux autres distinguent « rempli par l'application » de « choisi par la personne ». Celui-ci ne le distingue pas — **et c'est toute l'explication du point 5**. C'est la dixième fois recensée dans ce dépôt qu'un mécanisme est posé sur une porte et pas sur sa jumelle ; la famille est écrite dans `BUGS.md` (§59), et elle a la propriété gênante de ne se voir **ni à la relecture ni au banc d'essai**.

Une décision antérieure a relire avant tout correctif

Le dépôt porte une règle : *un retrait volontaire doit être écrit, sinon il redevient un bug*. Il y en a une ici, et je la signale plutôt que de la contourner. **ft-v1105**, dans le code, en toutes lettres :

```
ON NE RETIRE PAS LE PRE-REPLISSAGE : sans lui on retombe à 100 g,
ce qui est pire, et le chemin rapide disparaît.
On ne cache pas le nombre, on lui rend sa source.
```

ELLE NE CONTREDIT PAS TA REGLE — ELLE PARLE D'AUTRE CHOSE

ft-v1105 parle de ce qui est **AFFICHE**. Ta regle parle de ce qui est **ENREGISTRE**. **Les deux exigences peuvent tenir ensemble** : un drapeau laisse l'écran exactement comme il est, avec son nombre et sa source écrite à côté, et empêche seulement l'écriture tant qu'aucun geste n'a eu lieu. Et le commentaire de ft-v1105 **nomme lui-même la jumelle qu'il n'a pas faite** : il cite la décision « *on donne le choix et pas imposer* » prise pour la quantité de la dernière fois, et note qu'elle n'avait pas été appliquée ici.

Ce qui existe déjà, si un correctif est décidé

Aucun mécanisme n'est à inventer. Les solutions que tu listes — champ vide, bouton « 205 g », bouton « 410 g » — sont déjà construites dans l'application, sur d'autres portes :

fonction existante	ce qu'elle fait déjà
<code>_bcProposerDerniere(q)</code>	vide le champ et affiche une pastille cliquable « la dernière fois » — <i>exactement le motif demande</i>
<code>_bcProposerPaquet()</code> / <code>_bcReprendrePaquet()</code>	affiche « 410 g (le paquet entier) » et l'applique au clic
<code>_afPoidsPose</code> / <code>_afPortionPose</code>	le drapeau de geste, sur les deux autres blocs

DEUX LECTURES POSSIBLES, ET C'EST MICHEL QUI TRANCHE

A — champ vide + pastilles : après un scan, la quantité est vide et deux boutons sont offerts (la portion fabricant, le paquet). Un clic de plus à chaque scan, et l'écran change. **B — écran inchangé, drapeau à l'écriture** : le nombre reste affiché avec sa source, mais q n'est écrit que si la personne a touché le champ ou cliqué une pastille. Zéro clic en plus, ft-v1105 intacte à la lettre. **Les deux satisfont ta règle** ; elles diffèrent sur le confort, pas sur la donnée.

PERIMETRE RESPECTE

Non traitées, comme demande : les kcal d'Open Food Facts, la cohérence énergie/macros, la visibilité de l'alerte, P1 et les portions nommées, la migration historique. **Et aucune ligne de production n'a été modifiée** — cette trace est une mesure, pas un correctif.